it may be necessary to know what tokens can legally come after the nonterminal *A*, since such tokens indicate that *A* may appropriately disappear at this point in the parse. This set is called the **Follow** set of *A*. The computation of this set is also made precise in Section 4.3.

A final concern that may require the computation of First and Follow sets is early error detection. Consider, for example, the calculator program of Figure 4.1. Given the input `)3-2)`, the parser will descend from `exp` to `term` to `factor` before an error is reported, while it would be possible to declare the error already in `exp`, since a right parenthesis is not a legal first character in an expression. The First set of *exp* would tell us this, allowing for earlier error detection. (Error detection and recovery is discussed in more detail at the end of the chapter.)

4.2 LL(1) PARSING

4.2.1 The Basic Method of LL(1) Parsing

LL(1) parsing uses an explicit stack rather than recursive calls to perform a parse. It is helpful to represent this stack in a standard way, so that the actions of an LL(1) parser can be visualized quickly and easily. In this introductory discussion, we use the very simple grammar that generates strings of balanced parentheses:

$$S \rightarrow (S) S \mid \varepsilon$$

(see Example 3.5 of the previous chapter).

Table 4.1 shows the actions of a top-down parser given this grammar and the string `()`. In this table are four columns. The first column numbers the steps for later reference. The second column shows the contents of the parsing stack, with the bottom of the stack to the left and the top of the stack to the right. We mark the bottom of the stack with a dollar sign. Thus, a stack containing the nonterminal *S* at the top appears as

$$\$ S$$

and additional stack items would be pushed on the right. The third column of Table 4.1 shows the input. The input symbols are listed from left to right. A dollar sign is used to mark the end of the input (this corresponds to an EOF token generated by a scanner). The fourth column of the table gives a shorthand description of the action taken by the parser, which will change the stack and (possibly) the input, as given in the next row of the table.

Table 4.1 Parsing actions of a top-down parser

	Parsing stack	Input	Action
1	$ *S*	( ) $	*S* → (*S*) *S*
2	$ *S*) *S* (	( ) $	match
3	$ *S*) *S*	) $	*S* → ε
4	$ *S*)	) $	match
5	$ *S*	$	*S* → ε
6	$	$	accept

A top-down parser begins by pushing the start symbol onto the stack. It accepts an input string if, after a series of actions, the stack and the input become empty. Thus, a general schematic for a successful top-down parse is

\$ *StartSymbol*	*InputString* \$
...	...
...	...
\$	\$ accept

In the case of our example, the start symbol is S, and the input string is `()`.

A top-down parser parses by replacing a nonterminal at the top of the stack by one of the choices in the grammar rule (in BNF) for that nonterminal. It does this with a view to producing the current input token on the top of the parsing stack, whence it has matched the input token and can discard it from both the stack and the input. These two actions,

1. Replace a nonterminal A at the top of the stack by a string α using the grammar rule choice $A \rightarrow \alpha$, and
2. Match a token on top of the stack with the next input token,

are the two basic actions in a top-down parser. The first action could be called **generate**; we indicate this action by writing the BNF choice used in the replacement (whose left-hand side must be the nonterminal currently at the top of the stack). The second action matches a token at the top of the stack with the next token in the input (and throws both away by popping the stack and advancing the input); we indicate this action by writing the word *match*. It is important to note that in the *generate* action, the replacement string α from the BNF must be pushed *in reverse* onto the stack (since that ensures that the string α will come to the top of the stack in left to right order).

For example, at step 1 of the parse in Table 4.1, the stack and input are

\$ S `()` \$

and the rule we use to replace S on the top of the stack is $S \rightarrow$ `(` S `)` S, so that the string `)` S `(` is pushed onto the stack to obtain

\$ S `)` S `(` `()` \$

Now we have generated the next input terminal, namely a left parenthesis, on the top of the stack, and we perform a *match* action to obtain the following situation:

\$ S `)` S `)` \$

The list of generating actions in Table 4.1 corresponds precisely to the steps in a leftmost derivation of the string `()`:

$$
\begin{aligned}
S &\Rightarrow (S)S && [S \rightarrow (\ S\ )\ S] \\
&\Rightarrow ()S && [S \rightarrow \varepsilon] \\
&\Rightarrow () && [S \rightarrow \varepsilon]
\end{aligned}
$$

This is characteristic of top-down parsing. If we want to construct a parse tree as the parse proceeds, we can add node construction actions as each nonterminal or terminal is pushed onto the stack. Thus, the root node of the parse tree (corresponding to the start symbol) is constructed at the beginning of the parse. And in step 2 of Table 4.1, when

S is replaced by (S) S, the nodes for each of the four replacement symbols are constructed as the symbols are pushed onto the stack and are connected as children to the node of S that they replace on the stack. To make this effective, we have to modify the stack to contain pointers to these constructed nodes, rather than simply the nonterminals or terminals themselves. Further on, we shall see how this process can be modified to yield a construction of the syntax tree instead of the parse tree.

4.2.2 The LL(1) Parsing Table and Algorithm

Using the parsing method just described, when a nonterminal A is at the top of the parsing stack, a decision must be made, based on the current input token (the lookahead), which grammar rule choice for A to use when replacing A on the stack. By contrast, no decision is necessary when a token is at the top of the stack, since it is either the same as the current input token, and a match occurs, or it isn't, and an error occurs.

We can express the choices that are possible by constructing an **LL(1) parsing table**. Such a table is essentially a two-dimensional array indexed by nonterminals and terminals containing production choices to use at the appropriate parsing step (including \$ to represent the end of the input). We call this table $M[N, T]$. Here N is the set of nonterminals of the grammar, T is the set of terminals or tokens (for convenience, we suppress the fact that \$ must be added to T), and M can be thought of as the table of "moves." We assume that the table $M[N, T]$ starts out with all its entries empty. Any entries that remain empty after the construction represent potential errors that may occur during a parse.

We add production choices to this table according to the following rules:

1. If $A \rightarrow \alpha$ is a production choice, and there is a derivation $\alpha \Rightarrow^* a\,\beta$, where a is a token, then add $A \rightarrow \alpha$ to the table entry $M[A, a]$.
2. If $A \rightarrow \alpha$ is a production choice, and there are derivations $\alpha \Rightarrow^* \varepsilon$ and $S\,\$ \Rightarrow^* \beta\,A\,a\,\gamma$, where S is the start symbol and a is a token (or \$), then add $A \rightarrow \alpha$ to the table entry $M[A, a]$.

The idea behind these rules is as follows. In rule 1, given a token a in the input, we wish to select a rule $A \rightarrow \alpha$ if α can produce an a for matching. In rule 2, if A derives the empty string (via $A \rightarrow \alpha$), and if a is a token that can legally come after A in a derivation, then we want to select $A \rightarrow \alpha$ to make A disappear. Note that a special case of rule 2 is when $\alpha = \varepsilon$.

These rules are difficult to implement directly, and in the next section we will develop algorithms for doing so, involving the First and Follow sets already mentioned. In extremely simple cases, however, these rules can be carried out by hand.

Consider as a first example the grammar for balanced parentheses used in the previous subsection. There is one nonterminal (S), three tokens (left parenthesis, right parenthesis, and \$), and two production choices. Since there is only one nonempty production for S, namely, $S \rightarrow$ (S) S, every string derivable from S must be either empty or begin with a left parenthesis, and this production choice is added to the entry $M[S, (]$ (and only there). This completes all the cases under rule 1. Since $S \Rightarrow$ (S) S, rule 2 applies with $\alpha = \varepsilon$, $\beta =$ (, $A = S$, $a =$), and $\gamma = S\,\$$, so $S \rightarrow \varepsilon$ is added to $M[S,)]$. Since $S\,\$ \Rightarrow^* S\,\$$ (the empty derivation), $S \rightarrow \varepsilon$ is also added to $M[S, \$]$. This

completes the construction of the LL(1) parsing table for this grammar, which we can write in the following form:

M[N, T]	(	)	$
S	$S \rightarrow (S) S$	$S \rightarrow \varepsilon$	$S \rightarrow \varepsilon$

To complete the LL(1) parsing algorithm, this table must give unique choices for each nonterminal-token pair. Thus, we state the following

Definition

A grammar is an **LL(1) grammar** if the associated LL(1) parsing table has at most one production in each table entry.

An LL(1) grammar cannot be ambiguous, since the definition implies that an unambiguous parse can be constructed using the LL(1) parsing table. Indeed, given an LL(1) grammar, a parsing algorithm that uses the LL(1) parsing table is given in Figure 4.2. This algorithm results in precisely those actions described in the example of the previous subsection.

While the algorithm of Figure 4.2 requires that the parsing table entries have at most one production each, it is possible to build disambiguating rules into the table construction for simple ambiguous cases such as the dangling else problem, in a similar way to recursive-descent.

Figure 4.2
Table-based LL(1) parsing algorithm

```
(* assumes $ marks the bottom of the stack and the end of the input *)
push the start symbol onto the top of the parsing stack ;
while the top of the parsing stack ≠ $ and the next input token ≠ $ do
  if the top of the parsing stack is terminal a
       and the next input token = a
  then (* match *)
    pop the parsing stack ;
    advance the input ;
  else if the top of the parsing is nonterminal A
       and the next input token is terminal a
       and parsing table entry M[A, a] contains
            production A → X1X2 . . . Xn
  then (* generate *)
    pop the parsing stack ;
    for i := n downto 1 do
      push Xi onto the parsing stack ;
  else error ;
if the top of the parsing stack = $
       and the next input token = $
then accept
else error ;
```

Consider, for example, the simplified grammar of if-statements (see Example 3.6 in Chapter 3):

statement → *if-stmt* | **other**
if-stmt → **if** (*exp*) *statement else-part*
else-part → **else** *statement* | ε
exp → **0** | **1**

Constructing the LL(1) parsing table gives the result shown in Table 4.2, where we do not list the parentheses terminals (or) in the table, since they do not cause any actions. (The construction of this table will be explained in detail in the next section.)

Table 4.2
LL(1) parsing table for (ambiguous) if-statements

M[N, T]	**if**	**other**	**else**	0	1	$
statement	*statement* → *if-stmt*	*statement* → **other**				
if-stmt	*if-stmt* → **if** (*exp*) *statement else-part*					
else-part			*else-part* → **else** *statement* *else-part* → ε			*else-part* → ε
exp				*exp* → **0**	*exp* → **1**	

In Table 4.2, the entry M[*else-part*, **else**] contains two entries, corresponding to the dangling else ambiguity. As in recursive-descent, when constructing this table, we could apply a disambiguating rule that would always prefer the rule that generates the current lookahead token over any other, and thus the production

else-part → **else** *statement*

would be preferred over the production *else-part* → ε. This corresponds in fact to the most closely nested disambiguating rule. With this modification, Table 4.2 becomes unambiguous, and the grammar can be parsed as if it were an LL(1) grammar. For example, Table 4.3 shows the parsing actions of the LL(1) parsing algorithm, given the string

```
if(0) if(1) other else other
```

(For conciseness, we use the following abbreviations in that figure: *statement* = *S*, *if-stmt* = *I*, *else-part* = *L*, *exp* = *E*, **if** = **i**, **else** = **e**, **other** = **o**.)

Table 4.3
LL(1) parsing actions for *if-statements using the most closely nested disambiguating rule*

Parsing stack	Input	Action
$ *S*	`i(0)i(1)o e o`$	*S* → *I*
$ *I*	`i(0)i(1)o e o`$	*I* → `i(` *E* `)` *S L*
$ *L S* `)` *E* `(` `i`	`i(0)i(1)o e o`$	match
$ *L S* `)` *E* `(`	`(0)i(1)o e o`$	match
$ *L S* `)` *E*	`0)i(1)o e o`$	*E* → `0`
$ *L S* `)` `0`	`0)i(1)o e o`$	match
$ *L S* `)`	`)i(1)o e o`$	match
$ *L S*	`i(1)o e o`$	*S* → *I*
$ *L I*	`i(1)o e o`$	*I* → `i(` *E* `)` *S L*
$ *L L S* `)` *E* `(` `i`	`i(1)o e o`$	match
$ *L L S* `)` *E* `(`	`(1)o e o`$	match
$ *L L S* `)` *E*	`1)o e o`$	*E* → `1`
$ *L L S* `)` `1`	`1)o e o`$	match
$ *L L S* `)`	`)o e o`$	match
$ *L L S*	`o e o`$	*S* → `o`
$ *L L* `o`	`o e o`$	match
$ *L L*	`e o`$	*L* → `e` *S*
$ *L S* `e`	`e o`$	match
$ *L S*	`o`$	*S* → `o`
$ *L* `o`	`o`$	match
$ *L*	$	*L* → ε
$	$	accept

4.2.3 Left Recursion Removal and Left Factoring

Repetition and choice in LL(1) parsing suffer from similar problems to those that occur in recursive-descent parsing, and for that reason we have not yet been able to give an LL(1) parsing table for the simple arithmetic expression grammar of previous sections. We solved these problems for recursive-descent using EBNF notation. We cannot apply the same ideas to LL(1) parsing; instead, we must rewrite the grammar within the BNF notation into a form that the LL(1) parsing algorithm can accept. The two standard techniques that we apply are **left recursion removal** and **left factoring**. We consider each of these techniques. It must be emphasized that there is no guarantee that the application of these techniques will turn a grammar into an LL(1) grammar, just as EBNF is not guaranteed to solve all problems in writing a recursive-descent parser. Nevertheless, they are very useful in most practical situations and have the advantage that their application can be automated, so that, assuming a successful result, an LL(1) parser can be automatically generated using them (see the Notes and References section).

Left Recursion Removal Left recursion is commonly used to make operations left associative, as in the simple expression grammar, where

$$exp \rightarrow exp\ addop\ term \mid term$$

makes the operations represented by *addop* left associative. This is the simplest case of left recursion, where there is only a single left recursive production choice. Only slightly more complex is the case when more than one choice is left recursive, which happens if we write out *addop*:

$$exp \rightarrow exp + term \mid exp - term \mid term$$

Both these cases involve **immediate left recursion**, where the left recursion occurs only within the production of a single nonterminal (like *exp*). A more difficult case is *indirect* left recursion, such as in the rules

$$A \rightarrow B\,b \mid \ldots$$
$$B \rightarrow A\,a \mid \ldots$$

Such rules almost never occur in actual programming language grammars, but we include a solution to this case for completeness. We consider immediate left recursion first.

CASE 1: Simple immediate left recursion

In this case the left recursion is present only in grammar rules of the form

$$A \rightarrow A\,\alpha \mid \beta$$

where α and β are strings of terminals and nonterminals and β does not begin with A. We saw in Section 3.2.3 that this grammar rule generates strings of the form $\beta\alpha^n$, for $n \geq 0$. The choice $A \rightarrow \beta$ is the base case, while $A \rightarrow A\,\alpha$ is the recursive case.

To remove the left recursion, we rewrite this grammar rule into two rules: one that generates β first and one that generates the repetitions of α, using right recursion instead of left recursion:

$$A \rightarrow \beta\,A'$$
$$A' \rightarrow \alpha\,A' \mid \varepsilon$$

Example 4.1

Consider again the left recursive rule from the simple expression grammar:

$$exp \rightarrow exp\ addop\ term \mid term$$

This is of the form $A \rightarrow A\,\alpha \mid \beta$, with $A = exp$, $\alpha = addop\ term$, and $\beta = term$. Rewriting this rule to remove left recursion, we obtain

$$exp \rightarrow term\ exp'$$
$$exp' \rightarrow addop\ term\ exp' \mid \varepsilon$$

§

CASE 2: General immediate left recursion

This is the case where we have productions of the form

$$A \rightarrow A\,\alpha_1 \mid A\,\alpha_2 \mid \ldots \mid A\,\alpha_n \mid \beta_1 \mid \beta_2 \mid \ldots \mid \beta_m$$

where none of the $\beta_1, \ldots, \beta_m$ begin with A. In this case, the solution is similar to the simple case, with the choices expanded accordingly:

$$A \to \beta_1 A' \mid \beta_2 A' \mid \ldots \mid \beta_m A'$$
$$A' \to \alpha_1 A' \mid \alpha_2 A' \mid \ldots \mid \alpha_n A' \mid \varepsilon$$

Example 4.2 Consider the grammar rule

$$exp \to exp + term \mid exp - term \mid term$$

We remove the left recursion as follows:

$$exp \to term\ exp'$$
$$exp' \to + term\ exp' \mid - term\ exp' \mid \varepsilon$$ §

CASE 3: General left recursion

The algorithm we describe here is guaranteed to work only in the case of grammars with no ε-productions and with no cycles, where a **cycle** is a derivation of at least one step that begins and ends with the same nonterminal: $A \Rightarrow \alpha \Rightarrow^* A$. A cycle is almost certain to cause a parser to go into an infinite loop, and grammars with cycles never appear as programming language grammars. Programming language grammars do have ε-productions, but usually in very restricted forms, so this algorithm will almost always work for these grammars too.

The algorithm works by picking an arbitrary order for all the nonterminals of the language, say, $A_1, \ldots, A_m$, and then eliminating all left recursion that does not increase the index of the A_i's. This eliminates all rules of the form $A_i \to A_j\ \gamma$ with $j \leq i$. If we do this for each i from 1 to m, then no recursive loop can be left, since every step in such a loop would only increase the index, and thus the original index cannot be reached again. The algorithm is given in detail in Figure 4.3.

Figure 4.3
Algorithm for general left recursion removal

for $i := 1$ **to** m **do**
  for $j := 1$ **to** $i-1$ **do**
    replace each grammar rule choice of the form $A_i \to A_j\ \beta$ *by the rule*
    $A_i \to \alpha_1\ \beta \mid \alpha_2\ \beta \mid \ldots \mid \alpha_k\ \beta$, *where* $A_j \to \alpha_1 \mid \alpha_2 \mid \ldots \mid \alpha_k$ *is the current rule for* A_j

Example 4.3 Consider the following grammar:

$$A \to B\ a \mid A\ a \mid c$$
$$B \to B\ b \mid A\ b \mid d$$

(This grammar is completely artificial, since this situation does not occur in any standard programming language.)

We consider B to have a higher number than A for purposes of the algorithm (that is, $A_1 = A$ and $A_2 = B$). Since $n = 2$, the outer loop of the algorithm of Figure 4.3 executes twice, once for $i = 1$ and once for $i = 2$. When $i = 1$, the inner loop (with index j) does not execute, so the only action is to remove the immediate left recursion of A. The resulting grammar is

$$\begin{aligned} A &\rightarrow B\,a\,A' \mid c\,A' \\ A' &\rightarrow a\,A' \mid \varepsilon \\ B &\rightarrow B\,b \mid A\,b \mid d \end{aligned}$$

Now the outer loop executes for $i = 2$, and the inner loop executes once, with $j = 1$. In this case we eliminate the rule $B \rightarrow A\,b$ by replacing A with its choices from the first rule. Thus, we obtain the grammar

$$\begin{aligned} A &\rightarrow B\,a\,A' \mid c\,A' \\ A' &\rightarrow a\,A' \mid \varepsilon \\ B &\rightarrow B\,b \mid B\,a\,A'\,b \mid c\,A'\,b \mid d \end{aligned}$$

Finally, we remove the immediate left recursion of B to obtain

$$\begin{aligned} A &\rightarrow B\,a\,A' \mid c\,A' \\ A' &\rightarrow a\,A' \mid \varepsilon \\ B &\rightarrow c\,A'\,b\,B' \mid d\,B' \\ B' &\rightarrow b\,B' \mid a\,A'\,b\,B' \mid \varepsilon \end{aligned}$$

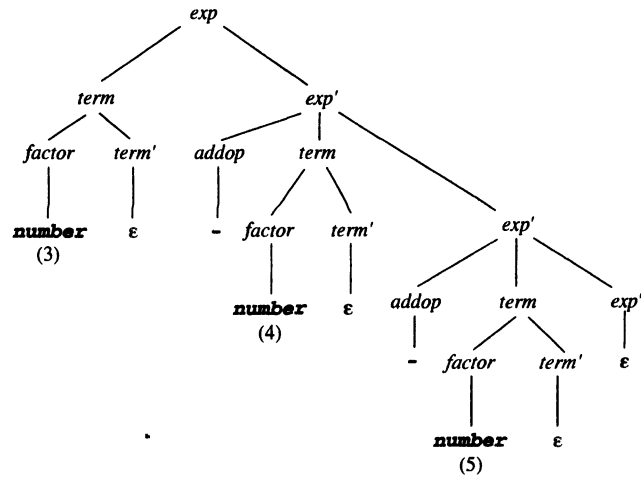
This grammar has no left recursion. §

Left recursion removal does not change the language being recognized, but it does change the grammar and thus also the parse trees. Indeed, this change causes a complication for the parser (and for the parser designer). Consider, for example, the simple expression grammar that we have been using as a standard example. As we saw, the grammar is left recursive, so as to express the left associativity of the operations. If we remove the immediate left recursion as in Example 4.1, we obtain the grammar given in Figure 4.4.

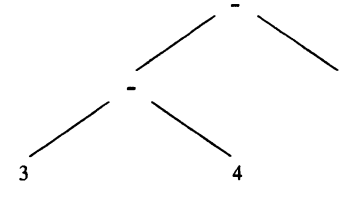
Figure 4.4
Simple arithmetic expression grammar with left recursion removed

$$\begin{aligned} &exp \rightarrow term\ exp' \\ &exp' \rightarrow addop\ term\ exp' \mid \varepsilon \\ &addop \rightarrow \texttt{+} \mid \texttt{-} \\ &term \rightarrow factor\ term' \\ &term' \rightarrow mulop\ factor\ term' \mid \varepsilon \\ &mulop \rightarrow \texttt{*} \\ &factor \rightarrow \texttt{(}\ exp\ \texttt{)} \mid \textbf{number} \end{aligned}$$

Now consider the parse tree for the expression **3-4-5**:



This tree no longer expresses the left associativity of subtraction. Nevertheless, a parser should still construct the appropriate left associative syntax tree:



It is not completely trivial to do this using the new grammar. To see how, consider instead the somewhat simpler question of computing the value of the expression using the given parse tree. To do this, the value 3 must be passed from the root *exp* node to its right child *exp'*. This *exp'* node must then subtract 4 and pass the new value -1 down to *its* rightmost child (another *exp'*). This *exp'* node in its turn must subtract 5 and pass the value -6 to the final *exp'* node. This node has an ε child only and simply passes the value -6 back. This value is then returned up the tree to the root *exp* node and is the final value of the expression.

Consider how this would work in a recursive-descent parser. The grammar with its left recursion removed would give rise to the procedures *exp* and *exp'* as follows:

```
procedure exp ;
begin
  term ;
  exp' ;
end exp ;
```

```
procedure exp' ;
begin
  case token of
   + : match (+) ;
       term ;
       exp' ;
   - : match (-) ;
       term ;
       exp' ;
  end case ;
end exp' ;
```

To get these procedures to actually compute the value of the expression, we would rewrite them as follows:

```
function exp : integer ;
var temp : integer ;
begin
  temp := term ;
  return exp'(temp) ;
end exp ;


function exp' ( valsofar : integer ) : integer ;
begin
  if token = + or token = - then
    case token of
     + : match (+) ;
         valsofar := valsofar + term ;
     - : match (-) ;
         valsofar := valsofar - term ;
    end case ;
    return exp'(valsofar) ;
  else return valsofar ;
end exp' ;
```

Note how the *exp'* procedure now needs a parameter passed from the *exp* procedure. A similar situation occurs if these procedures are to return a (left associative) syntax tree. The code we gave in Section 4.1 used a simpler solution based on EBNFs, which does not require the extra parameter.

Finally, we note that the new expression grammar of Figure 4.4 is indeed an LL(1) grammar. The LL(1) parsing table is given in Table 4.4. As with previous tables, we will return to its construction in the next section.

Left Factoring Left factoring is required when two or more grammar rule choices share a common prefix string, as in the rule

$$A \rightarrow \alpha\,\beta \mid \alpha\,\gamma$$

Table 4.4
LL(1) parsing table for the grammar of Figure 4.4

M[N, T]	(	number	)	+	-	*	$
exp	*exp* → *term exp'*	*exp* → *term exp'*					
exp'			*exp'* → ε	*exp'* → *addop term exp'*	*exp'* → *addop term exp'*		*exp'* → ε
addop				*addop* → +	*addop* → -		
term	*term* → *factor term'*	*term* → *factor term'*					
term'			*term'* → ε	*term'* → ε	*term'* → ε	*term'* → *mulop factor term'*	*term'* → ε
mulop						*mulop* → *	
factor	*factor* → (*exp*)	*factor* → **number**					

Examples are the right recursive rule for statement sequences (Example 3.7, of Chapter 3):

stmt-sequence → *stmt* ; *stmt-sequence* | *stmt*
stmt → **s**

and the following version of the if-statement:

if-stmt → **if** (*exp*) *statement*
| **if** (*exp*) *statement* **else** *statement*

Obviously, an LL(1) parser cannot distinguish between the production choices in such a situation. The solution in this simple case is to "factor" the α out on the left and rewrite the rule as two rules

$$A \rightarrow \alpha A'$$
$$A' \rightarrow \beta \mid \gamma$$

(If we wanted to use parentheses as metasymbols in grammar rules, we could also write $A \rightarrow \alpha\ (\beta \mid \gamma)$, which looks exactly like factoring in arithmetic.) For left factoring to work properly, we must make sure that α is in fact the longest string shared by the right-hand sides. It is also possible for there to be more than two options that share a prefix. We state the general algorithm in Figure 4.5 and then apply it to a number of examples. Note that as the algorithm proceeds, the number of production choices for each nonter-

minal that can share a prefix is reduced by at least one at each step, so the algorithm is guaranteed to terminate.

Figure 4.5
Algorithm for left factoring a grammar

while *there are changes to the grammar* **do**
  for *each nonterminal A* **do**
    let α *be a prefix of maximal length that is shared by two or more production choices for A*
  if $\alpha \neq \varepsilon$ **then**
    let $A \rightarrow \alpha_1 \mid \alpha_2 \mid \ldots \mid \alpha_n$ *be all the production choices for A and suppose that* $\alpha_1, \ldots, \alpha_k$ *share* α, *so that* $A \rightarrow \alpha\,\beta_1 \mid \ldots \mid \alpha\,\beta_k \mid \alpha_{k+1} \mid \ldots \mid \alpha_n$, *the* β_j*'s share no common prefix, and the* $\alpha_{k+1}, \ldots, \alpha_n$ *do not share* α
    replace the rule $A \rightarrow \alpha_1 \mid \alpha_2 \mid \ldots \mid \alpha_n$ *by the rules*
    $A \rightarrow \alpha\,A' \mid \alpha_{k+1} \mid \ldots \mid \alpha_n$
    $A' \rightarrow \beta_1 \mid \ldots \mid \beta_k$

Example 4.4

Consider the grammar for statement sequences, written in right recursive form:

stmt-sequence → *stmt* **;** *stmt-sequence* | *stmt*
stmt → **s**

The grammar rule for *stmt-sequence* has a shared prefix that can be left factored as follows:

stmt-sequence → *stmt stmt-seq'*
stmt-seq' → **;** *stmt-sequence* | ε

Notice that if we had written the *stmt-sequence* rule left recursively instead of right recursively,

stmt-sequence → *stmt-sequence* **;** *stmt* | *stmt*

then removing the immediate left recursion would result in the rules

stmt-sequence → *stmt stmt-seq'*
stmt-seq' → **;** *stmt stmt-seq'* | ε

This is almost identical to the result we obtained from left factoring, and indeed making the substitution of *stmt-sequence* for *stmt stmt-seq'* in the last rule makes the two results identical. §

Example 4.5

Consider the following (partial) grammar for if-statements:

if-stmt → **if** **(** *exp* **)** *statement*
  | **if** **(** *exp* **)** *statement* **else** *statement*

The left factored form of this grammar is

$$\textit{if-stmt} \rightarrow \texttt{if (}\ \textit{exp}\ \texttt{)}\ \textit{statement}\ \textit{else-part}$$
$$\textit{else-part} \rightarrow \texttt{else}\ \textit{statement} \mid \varepsilon$$

This is precisely the form in which we used it in Section 4.2.2 (see Table 4.2). §

Example 4.6 Suppose we wrote an arithmetic expression grammar in which we gave an arithmetic operation right associativity instead of left associativity (we use + here just to have a concrete example):

$$\textit{exp} \rightarrow \textit{term} + \textit{exp} \mid \textit{term}$$

This grammar needs to be left factored, and we obtain the rules

$$\textit{exp} \rightarrow \textit{term}\ \textit{exp}'$$
$$\textit{exp}' \rightarrow + \textit{exp} \mid \varepsilon$$

Now, continuing as in Example 4.4, suppose we substitute *term exp'* for *exp* in the second rule (this is legal, since this expansion would take place at the next step in a derivation anyway). We then obtain

$$\textit{exp} \rightarrow \textit{term}\ \textit{exp}'$$
$$\textit{exp}' \rightarrow + \textit{term}\ \textit{exp}' \mid \varepsilon$$

This is identical to the grammar obtained from the left recursive rule by left recursion removal. Thus, both left factoring and left recursion removal can obscure the semantics of the language structure (in this case, they both obscure the associativity).

If, for example, we wished to preserve the right associativity of the operation from the above grammar rules (in either form), then we must arrange that each + operation is applied at the end rather than at the beginning. We leave it to the reader to write this out for recursive-descent procedures. §

Example 4.7 Here is a typical case where a programming language grammar fails to be LL(1), because procedure calls and assignments both begin with an identifier. We write out the following representation of this problem:

$$\textit{statement} \rightarrow \textit{assign-stmt} \mid \textit{call-stmt} \mid \texttt{other}$$
$$\textit{assign-stmt} \rightarrow \texttt{identifier :=}\ \textit{exp}$$
$$\textit{call-stmt} \rightarrow \texttt{identifier (}\ \textit{exp-list}\ \texttt{)}$$

This grammar is not LL(1) because **identifier** is shared as the first token of both *assign-stmt* and *call-stmt* and, thus, could be the lookahead token for either. Unfortunately, the grammar is not in a form that can be left factored. What we must do is first replace *assign-stmt* and *call-stmt* by the right-hand sides of their defining productions, as follows:

$$\begin{aligned} \textit{statement} \rightarrow\ & \texttt{identifier} := \textit{exp} \\ & \mid \texttt{identifier} \texttt{(} \textit{exp-list} \texttt{)} \\ & \mid \texttt{other} \end{aligned}$$

Then we left factor to obtain

$$\begin{aligned} \textit{statement} &\rightarrow \texttt{identifier}\ \textit{statement}' \\ &\quad \mid \texttt{other} \\ \textit{statement}' &\rightarrow := \textit{exp} \mid \texttt{(}\ \textit{exp-list}\ \texttt{)} \end{aligned}$$

Note how this obscures the semantics of call and assignment by separating the identifier (the variable to be assigned to or the procedure to be called) from the actual call or assign action (represented by *statement'*). An LL(1) parser must fix this up by making the identifier available to the call or assign step in some way (as a parameter, say) or by otherwise adjusting the syntax tree. §

Finally, we note that all the examples to which we have applied left factoring do indeed become LL(1) grammars after the transformation. We will construct the LL(1) parsing tables for some of them in the next section. Others are left for the exercises.

4.2.4 Syntax Tree Construction in LL(1) Parsing

It remains to comment on how LL(1) parsing may be adapted to construct syntax trees rather than parse trees. (We described in Section 4.2.1 how parse trees can be constructed using the parsing stack.) We saw in the section on recursive descent parsing that adapting that method to syntax tree construction was relatively easy. By contrast, LL(1) parsers are more difficult to adapt. This is partially because, as we have seen, the structure of the syntax tree (such as left associativity) can be obscured by left factoring and left recursion removal. Mainly, however, it is due to the fact that the parsing stack represents only predicted structures, not structures that have been actually seen. Thus, the construction of syntax tree nodes must be delayed to the point when structures are removed from the parsing stack, rather than when they are first pushed. In general, this requires that an extra stack be used to keep track of syntax tree nodes and that "action" markers be placed in the parsing stack to indicate when and what actions on the tree stack should occur. Bottom-up parsers (the next chapter) are easier to adapt to the construction of syntax trees using a parsing stack and are, therefore, preferred as table-driven stack-based parsing methods. Thus, we give only a brief example of the details of how this can be done for LL(1) parsing.

Example 4.8 We use as our grammar a barebones expression grammar with only an addition operation. The BNF is

$$E \rightarrow E + \texttt{n} \mid \texttt{n}$$

This causes addition to be applied left associatively. The corresponding LL(1) grammar with left recursion removed is

$$E \rightarrow \mathbf{n}\, E'$$
$$E' \rightarrow \mathbf{+}\, \mathbf{n}\, E' \mid \varepsilon$$

We show now how this grammar can be used to compute the arithmetic value of the expression. Construction of a syntax tree is similar.

To compute a value for the result of an expression, we will use a separate stack to store the intermediate values of the computation, which we call the **value stack**. We must schedule two operations on that stack. The first operation is a push of a number when it is matched in the input. The second is the addition of two numbers in the stack. The first can be performed by the *match* procedure (based on the token it is matching). The second needs to be scheduled on the parsing stack. We will do this by pushing a special symbol on the parsing stack, which, when popped, will indicate that an addition is to be performed. The symbol that we use for this purpose is the pound sign (#). This symbol now becomes a new stack symbol and must also be added to the grammar rule that matches a +, namely, the rule for E':

$$E' \rightarrow \mathbf{+}\, \mathbf{n}\, \# E' \mid \varepsilon$$

Note that the addition is scheduled just *after* the next number, but before any more E' nonterminals are processed. This guarantees left associativity. Now let us see how the computation of the value of the expression **3+4+5** takes place. We indicate the parsing stack, input, and action as before, but list the value stack on the right (it grows toward the left). The actions of the parser are given in Table 4.5.

Table 4.5
Parsing stack with value stack actions for Example 4.8

Parsing stack	Input	Action	Value stack
$ E	3 + 4 + 5 $	$E \rightarrow \mathbf{n}\, E'$	$
$ E' **n**	3 + 4 + 5 $	match/push	$
$ E'	+ 4 + 5 $	$E' \rightarrow \mathbf{+}\, \mathbf{n}\, \# E'$	3 $
$ E' # **n** +	+ 4 + 5 $	match	3 $
$ E' # **n**	4 + 5 $	match/push	3 $
$ E' #	+ 5 $	addstack	4 3 $
$ E'	+ 5 $	$E' \rightarrow \mathbf{+}\, \mathbf{n}\, \# E'$	7 $
$ E' # **n** +	+ 5 $	match	7 $
$ E' # **n**	5 $	match/push	7 $
$ E' #	$	addstack	5 7 $
$ E'	$	$E' \rightarrow \varepsilon$	12 $
$	$	accept	12 $

Note that when an addition takes place, the operands are in reverse order on the value stack. This is typical of such stack-based evaluation schemes. §

4.3 FIRST AND FOLLOW SETS

To complete the LL(1) parsing algorithm, we develop an algorithm to construct the LL(1) parsing table. As we have already indicated at several points, this involves computing the First and Follow sets, and we turn in this section to the definition and construction of these sets, after which we give a precise description of the construction of an LL(1) parsing table. At the end of this section we briefly consider how the construction can be extended to more than one symbol of lookahead.

4.3.1 First Sets

Definition

Let X be a grammar symbol (a terminal or nonterminal) or ε. Then the set **First(X)** consisting of terminals, and possibly ε, is defined as follows.

1. If X is a terminal or ε, then First(X) = $\{X\}$.
2. If X is a nonterminal, then for each production choice $X \to X_1X_2 \ldots X_n$, First(X) contains First(X_1) $- \{\varepsilon\}$. If also for some $i < n$, all the sets First(X_1), . . . , First(X_i) contain ε, then First(X) contains First(X_{i+1}) $- \{\varepsilon\}$. If all the sets First(X_1), . . . , First(X_n) contain ε, then First(X) also contains ε.

Now define **First(α)** for any string $\alpha = X_1X_2 \ldots X_n$ (a string of terminals and nonterminals), as follows. First(α) contains First(X_1) $- \{\varepsilon\}$. For each $i = 2, \ldots, n$, if First(X_k) contains ε for all $k = 1, \ldots, i - 1$, then First(α) contains First(X_i) $- \{\varepsilon\}$. Finally, if for all $i = 1, \ldots, n$, First(X_i) contains ε, then First(α) contains ε.

This definition can be easily turned into an algorithm. Indeed, the only difficult case is to compute First(A) for every nonterminal A, since the First set of a terminal is trivial and the first set of a string α is built up from the first sets of individual symbols in at most n steps, where n is the number of symbols in α. We thus state the pseudocode for the algorithm only in the case of nonterminals, and this is given in Figure 4.6.

Figure 4.6
Algorithm for computing First(A) for all nonterminals A

```
for all nonterminals A do First(A) := {};
while there are changes to any First(A) do
  for each production choice A → X1X2 . . . Xn do
    k := 1 ; Continue := true ;
    while Continue = true and k <= n do
      add First(Xk)−{ε} to First(A);
      if ε is not in First(Xk) then Continue := false ;
      k := k + 1 ;
    if Continue = true then add ε to First(A) ;
```

It is also easy to see how this definition can be interpreted in the absence of ε-productions: simply keep adding First(X_1) to First(A) for each nonterminal A and production choice $A \to X_1 \ldots$ until no further additions take place. In other words, we consider only the case $k = 1$ in Figure 4.6, and the inner while-loop is not needed. We state this algorithm separately in Figure 4.7. In the presence of ε-productions, the situation is more complicated, since we must also ask whether ε is in First(X_1), and if so continuing with the same process for X_2, and so on. The process will still finish after finitely many steps, however. Indeed, not only does this process compute the terminals that may appear as the first symbols in a string derived from a nonterminal, but it also determines whether a nonterminal can derive the empty string (i.e., disappear). Such nonterminals are called nullable:

Figure 4.7
Simplified algorithm of Figure 4.6 in the absence of ε-productions

```
for all nonterminals A do First(A) := {};
while there are changes to any First(A) do
  for each production choice A → X1X2 ... Xn do
    add First(X1) to First(A);
```

Definition

A nonterminal A is **nullable** if there exists a derivation $A \Rightarrow^* \varepsilon$.

Now we show the following theorem.

Theorem

A nonterminal A is nullable if and only if First(A) contains ε.

Proof: We show that if A is nullable, then First(A) contains ε. The converse can be proved in a similar manner. We use induction on the length of a derivation. If $A \Rightarrow \varepsilon$, then there must be a production $A \to \varepsilon$, and by definition, First(A) contains First(ε) = $\{\varepsilon\}$. Assume now the truth of the statement for derivations of length $< n$, and let $A \Rightarrow X_1 \ldots X_k \Rightarrow^* \varepsilon$ be a derivation of length n (using a production choice $A \to X_1 \ldots X_k$). If any of the X_i are terminals, they cannot derive ε, so all the X_i must be nonterminals. Indeed, the existence of the above derivation of which they are a part implies that each $X_i \Rightarrow^* \varepsilon$, and in fewer than n steps. Thus, by the induction assumption, for each i, First(X_i) contains ε. Finally, by definition, First(A) must contain ε

We give several examples of the computation of First sets for nonterminals.

Example 4.9 Consider our simple integer expression grammar:[2]

exp → *exp addop term* | *term*
addop → **+** | **-**
term → *term mulop factor* | *factor*
mulop → *
factor → **(** *exp* **)** | **number**

We write out each choice separately so that we may consider them in order (we also number them for reference):

(1) *exp* → *exp addop term*
(2) *exp* → *term*
(3) *addop* → **+**
(4) *addop* → **-**
(5) *term* → *term mulop factor*
(6) *term* → *factor*
(7) *mulop* → *
(8) *factor* → **(** *exp* **)**
(9) *factor* → **number**

This grammar contains no ε-productions, so we may use the simplified algorithm of Figure 4.7. We also note that the left recursive rules 1 and 5 will add nothing to the computation of First sets.[3] For example, grammar rule 1 states only that First(*exp*) should be added to First(*exp*). Thus, we could delete these productions from the computation. In this example, however, we will retain them in the list for clarity.

Now we apply the algorithm of Figure 4.7, considering the productions in the order just given. Production 1 makes no changes. Production 2 adds the contents of First(*term*) to First(*exp*). But First(*term*) is currently empty, so this also changes nothing. Rules 3 and 4 add **+** and **-** to First(*addop*), respectively, so First(*addop*) = {**+**, **-**}. Rule 5 adds nothing. Rule 6 adds First(*factor*) to First(*term*), but First(*factor*) is currently still empty, so no change occurs. Rule 7 adds * to First(*mulop*) so First(*mulop*) = {*}. Rule 8 adds **(** to First(*factor*), and rule 9 adds **number** to First(*factor*), so First(*factor*) = { **(**, **number** }. We now begin again with rule 1, since there have been changes. Now rules 1 through 5 make no changes (First(*term*) is still empty). Rule 6 adds First(*factor*) to First(*term*), and First(*factor*) = { **(**, **number** }, so now also First(*term*) = { **(**, **number** }. Rules 8 and 9 make no more changes. Again, we must begin with rule 1, since one set has changed. Rule 2 will finally add the new contents of First(*term*) to First(*exp*), and First(*exp*) = { **(**, **number** }. One more pass through the grammar rules is necessary, and no more changes occur, so after four passes we have computed the following First sets:

2. This grammar has left recusion and is not LL(1), so we will not be able to build an LL(1) parsing table for it. However, it is still a useful example of how to compute First sets.

3. In the presence of ε-productions, left recursive rules may contribute to First sets.

First(*exp*) = { (, **number** }
First(*term*) = { (, **number** }
First(*factor*) = { (, **number** }
First(*addop*) = {+, -}
First(*mulop*) = {*}

(Note that if we had listed the grammar rules for *factor* first instead of last, we could have reduced the number of passes from four to two.) We indicate this computation in Table 4.6. In that table, only changes are recorded, in the appropriate box where they occur. Blank entries indicate that no set has changed at that step. We also suppress the last pass, since no changes occur.

Table 4.6
Computation of First sets for the grammar of Example 4.9

Grammar rule	Pass 1	Pass 2	Pass 3
exp → *exp* *addop term*			
exp → *term*			First(*exp*) = { (, **number** }
addop → +	First(*addop*) = {+}		
addop → -	First(*addop*) = {+, -}		
term → *term* *mulop factor*			
term → *factor*		First(*term*) = { (, **number** }	
mulop → *	First(*mulop*) = {*}		
factor → (*exp*)	First(*factor*) = { (}		
factor → **number**	First(*factor*) = { (, **number** }		

§

Example 4.10 Consider the (left-factored) grammar of if-statements (Example 4.5):

statement → *if-stmt* | **other**
if-stmt → **if** (*exp*) *statement else-part*
else-part → **else** *statement* | ε
exp → **0** | **1**

This grammar does have an ε-production, but only the *else-part* nonterminal is nullable, so the complications to the computation are minimal. Indeed, we will only need to add ε at one step, and all the other steps will remain unaffected, since none

begin with a nonterminal whose First set contains ε. This is fairly typical for actual programming language grammars, where the ε-productions are almost always very limited and seldom exhibit the complexity of the general case.

As before, we write out the grammar rule choices separately and number them:

(1) *statement* → *if-stmt*
(2) *statement* → **other**
(3) *if-stmt* → **if** (*exp*) *statement else-part*
(4) *else-part* → **else** *statement*
(5) *else-part* → ε
(6) *exp* → **0**
(7) *exp* → **1**

Again, we proceed to step through the production choices one by one, making a new pass whenever a First set has changed in the previous pass. Grammar rule 1 begins by making no changes, since First(*if-stmt*) is still empty. Rule 2 adds the terminal **other** to First(*statement*), so First(*statement*) = {**other**}. Rule 3 adds **if** to First(*if-stmt*), so First(*if-stmt*) = {**if**}. Rule 4 adds **else** to First(*else-part*), so First(*else-part*) = {**else**}. Rule 5 adds ε to First(*else-part*), so First(*else-part*) = {**else**, ε}. Rules 6 and 7 add **0** and **1** to First(*exp*), so First(*exp*) = {**0**, **1**}. Now we make a second pass, beginning with rule 1. This rule now adds **if** to First(*statement*), since First(*if-stmt*) contains this terminal. Thus, First(*statement*) = {**if**, **other**}. No other changes occur during the second pass, and a third pass results in no changes whatsoever. Thus, we have computed the following First sets:

First(*statement*) = {**if**, **other**}
First(*if-stmt*) = {**if**}
First(*else-part*) = {**else**, ε}
First(*exp*) = {**0**, **1**}

Table 4.7 displays this computation in a similar manner to Table 4.6. As before, the table displays only changes, and the final pass (where no changes occur) is not displayed.

Table 4.7 Computation of First sets for the grammar of Example 4.10

Grammar rule	Pass 1	Pass 2
statement → *if-stmt*		First(*statement*) = {**if**, **other**}
statement → **other**	First(*statement*) = {**other**}	
if-stmt → **if** (*exp*) *statement else-part*	First(*if-stmt*) = {**if**}	
else-part → **else** *statement*	First(*else-part*) = {**else**}	
else-part → ε	First(*else-part*) = {**else**, ε}	
exp → **0**	First(*exp*) = {**0**}	
exp → **1**	First(*exp*) = {**0**, **1**}	

§

Example 4.11 Consider the following grammar for statement sequences (see Example 4.4):

$$\begin{aligned} &\textit{stmt-sequence} \rightarrow \textit{stmt stmt-seq}' \\ &\textit{stmt-seq}' \rightarrow \texttt{;}\ \textit{stmt-sequence} \mid \varepsilon \\ &\textit{stmt} \rightarrow \texttt{s} \end{aligned}$$

Again, we list the production choices individually:

(1) $\textit{stmt-sequence} \rightarrow \textit{stmt stmt-seq}'$
(2) $\textit{stmt-seq}' \rightarrow \texttt{;}\ \textit{stmt-sequence}$
(3) $\textit{stmt-seq}' \rightarrow \varepsilon$
(4) $\textit{stmt} \rightarrow \texttt{s}$

On the first pass rule 1 adds nothing. Rules 2 and 3 result in First(*stmt-seq'*) = {;, ε}. Rule 4 results in First(*stmt*) = {s}. On the second pass, rule 1 now results in First(*stmt-sequence*) = First(*stmt*) = {s}. No other changes are made, and a third pass also results in no changes. We have computed the following First sets:

First(*stmt-sequence*) = {s}
First(*stmt*) = {s}
First(*stmt-seq'*) = {;, ε}

We leave it to the reader to construct a table similar to those in Tables 4.6 and 4.7. §

4.3.2 Follow Sets

Definition

Given a nonterminal A, the set **Follow(A)**, consisting of terminals, and possibly \$, is defined as follows.

1. If A is the start symbol, then \$ is in Follow(A).
2. If there is a production $B \rightarrow \alpha A \gamma$, then First($\gamma$) − {ε} is in Follow($A$).
3. If there is a production $B \rightarrow \alpha A \gamma$ such that ε is in First (γ), then Follow(A) contains Follow(B).

We first examine the content of this definition and then write down the algorithm for the computation of the Follow sets that results from it. The first thing to notice is that the \$, used to mark the end of the input, behaves as if it were a token in the computation of the Follow set. Without it, we would not have a symbol to follow the entire string to be matched. Since such a string is generated by the start symbol of the grammar, the \$ must always be added to the Follow set of the start symbol. (It will be the only member of the Follow set of the start symbol if the start symbol never appears on the right-hand side of a production.)

The second thing to notice is that the empty "pseudotoken" ε is never an element of a Follow set. This makes sense because ε was used in First sets only to mark those strings that can disappear. It cannot actually be recognized in the input. Follow sym-

bols, on the other hand, will always be matched against existing input tokens (including the $ symbol, which will match an EOF generated by the scanner).

We also note that Follow sets are defined only for nonterminals, while First sets are also defined for terminals and for strings of terminals and nonterminals. We *could* extend the definition of Follow sets to strings of symbols, but it is unnecessary, since in constructing LL(1) parsing tables we will only need the Follow sets of nonterminals.

Finally, we note that the definition of Follow sets works "on the right" in productions, while the definition of the First sets works "on the left." By this we mean that a production $A \rightarrow \alpha$ has *no* information about the Follow set of A if α does not contain A. Only appearances of A on right-hand sides of productions will contribute to Follow(A). As a consequence, each grammar rule choice will in general make contributions to the Follow sets of *each* nonterminal that appears in the right-hand side, unlike the case of First sets, where each grammar rule choice adds to the First set of only one nonterminal (the one on the left).

Furthermore, given a grammar rule $A \rightarrow \alpha B$, Follow(B) will include Follow(A) by case (3) of the definition. This is because, in any string that contains A, A could be replaced by αB (this is "context-freeness" in action). This property is, in a sense, the opposite of the situation for First sets, where if $A \rightarrow B \alpha$, then First(A) includes First(B) (except possibly for ε).

In Figure 4.8 we give the algorithm for the computation of the Follow sets that results from the definition of Follow sets. We use this algorithm to compute Follow sets for the same three grammars for which we have just computed First sets. (As with the First sets, in the absence of ε-productions, the algorithm simplifies. We leave the simplification to the reader.)

Figure 4.8 Algorithm for the computation of Follow sets

```
Follow(start-symbol) := {$} ;
for all nonterminals A ≠ start-symbol do Follow(A) := {} ;
while there are changes to any Follow sets do
  for each production A → X1X2 . . . Xn do
    for each Xi that is a nonterminal do
      add First(Xi+1Xi+2 . . . Xn) − {ε} to Follow(Xi)
      (* Note: if i=n, then Xi+1Xi+2 . . . Xn = ε *)
      if ε is in First(Xi+1Xi+2 . . . Xn) then
        add Follow(A) to Follow(Xi)
```

Example 4.12 Consider again the simple expression grammar whose First sets we computed in Example 4.9, as follows:

First(*exp*) = { (, **number** }
First(*term*) = { (, **number** }
First(*factor*) = { (, **number** }
First(*addop*) = {+, -}
First(*mulop*) = {*}

We again write out the production choices with numbering:

(1) *exp* → *exp addop term*
(2) *exp* → *term*
(3) *addop* → +
(4) *addop* → -
(5) *term* → *term mulop factor*
(6) *term* → *factor*
(7) *mulop* → *
(8) *factor* → (*exp*)
(9) *factor* → **number**

Rules 3, 4, 7, and 9 all have no nonterminals on their right-hand sides, so they add nothing to the computation of Follow sets. We consider the other rules in order. Before we begin we set Follow(*exp*) = {$}; all other Follow sets are initialized to empty.

Rule 1 affects the Follow sets of three nonterminals: *exp*, *addop*, and *term*. First(*addop*) is added to Follow(*exp*), so now Follow(*exp*) = {$, +, -}. Next, First(*term*) is added to Follow(*addop*), so Follow(*addop*) = { (, **number**}. Finally, Follow(*exp*) is added to Follow(*term*), so Follow(*term*) = {$, +, -}.

Rule 2 again causes Follow(*exp*) to be added to Follow(*term*), but this has just been done by rule 1, so no change to the Follow sets occurs.

Rule 5 has three effects. First(*mulop*) is added to Follow(*term*), so Follow(*term*) = {$, +, -, *}. Then First(*factor*) is added to Follow(*mulop*), so Follow(*mulop*) = { (, **number**}. Finally, Follow(*term*) is added to Follow(*factor*), so Follow(*factor*) = {$, +, -, *}.

Rule 6 has the same effect as the last step for rule 5, and so makes no changes.

Finally, rule 8 adds First()) = {)} to Follow(*exp*), so Follow(*exp*) = {$, +, -,)}.

On the second pass, rule 1 adds) to Follow(*term*) (so Follow(*term*) = {$, +, -, *,)}), and rule 5 adds) to Follow(*factor*) (so Follow(*factor*) = {$, +, -, *,)}). A third pass results in no further changes, and the algorithm finishes. We have computed the following Follow sets:

Follow(*exp*) = {$, +, -,)}
Follow(*addop*) = { (, **number** }
Follow(*term*) = {$, +, -, *,)}
Follow(*mulop*) = { (, **number** }
Follow(*factor*) = {$, +, -, *,)}

As in the computation of First sets, we display the progress of this computation in Table 4.8. As before, we omit the concluding pass in this table and only indicate changes to the Follow sets when they occur. We also omit the four grammar rule choices that have no possibility of affecting the computation. (We include the two rules *exp* → *term* and *term* → *factor* because they have a potential effect, even though they have no actual effect.)

Table 4.8
Computation of Follow sets for the grammar of Example 4.12

Grammar rule	Pass 1	Pass 2
exp → *exp addop term*	Follow(*exp*) = {$, +, -} Follow(*addop*) = {(, **number**} Follow(*term*) = {$, +, -}	Follow(*term*) = {$, +, -, *,)}
exp → *term*		
term → *term mulop factor*	Follow(*term*) = {$, +, -, *} Follow(*mulop*) = {(, **number**} Follow(*factor*) = {$, +, -, *}	Follow(*factor*) = {$, +, -, *,)}
term → *factor*		
factor → **(** *exp* **)**	Follow(*exp*) = {$, +, -,)}	

§

Example 4.13 Consider again the simplified grammar of if-statements, whose First sets we computed in Example 4.10, as follows:

First(*statement*) = {**if**, **other**}
First(*if-stmt*) = {**if**}
First(*else-part*) = {**else**, ε}
First(*exp*) = {**0**, **1**}

We repeat here the production choices with numbering:

(1) *statement* → *if-stmt*
(2) *statement* → **other**
(3) *if-stmt* → **if** **(** *exp* **)** *statement else-part*
(4) *else-part* → **else** *statement*
(5) *else-part* → ε
(6) *exp* → **0**
(7) *exp* → **1**

Rules 2, 5, 6, and 7 have no effect on the computation of Follow sets, so we ignore them.

We begin by setting Follow(*statement*) = {$} and initializing Follow of the other nonterminals to empty. Rule 1 now adds Follow(*statement*) to Follow(*if-stmt*), so Follow(*if-stmt*) = {$}. Rule 3 affects the Follow sets of *exp*, *statement*, and *else-part*. First, Follow(*exp*) gets First( )) = {) }, so Follow(*exp*) = {) }. Then Follow(*statement*) gets First(*else-part*) − {ε}, so Follow(*statement*) = {$, **else**}. Finally, Follow (*if-stmt*) is added to both Follow(*else-part*) and Follow(*statement*) (this is because

else-part can disappear). The first addition gives Follow(*else-part*) = {$}, but the second results in no change. Finally, rule 4 causes Follow(*else-part*) to be added to Follow(*statement*), also resulting in no change.

In the second pass, rule 1 again adds Follow(*statement*) to Follow(*if-stmt*), resulting in Follow(*if-stmt*) = {$, **else**}. Rule 3 now causes the terminal **else** to be added to Follow(*else-part*), so Follow(*else-part*) = {$, **else**}. Finally, rule 4 causes no additional changes. The third pass also results in no more changes, and we have computed the following Follow sets:

Follow(*statement*) = {$, **else**}
Follow(*if-stmt*) = {$, **else**}
Follow(*else-part*) = {$, **else**}
Follow(*exp*) = { **)** }

We leave it to the reader to construct a table of this computation as in the previous example. §

Example 4.14 We compute the Follow sets for the simplified statement sequence grammar of Example 4.11, with the grammar rule choices:

(1) *stmt-sequence* → *stmt stmt-seq'*
(2) *stmt-seq'* → **;** *stmt-sequence*
(3) *stmt-seq'* → ε
(4) *stmt* → **s**

In Example 4.11, we computed the following First sets:

First(*stmt-sequence*) = {**s**}
First(*stmt*) = {**s**}
First(*stmt-seq'*) = {**;**, ε}

Grammar rules 3 and 4 make no contribution to the Follow set computation. We start with Follow(*stmt-sequence*) = {$} and the other Follow sets empty. Rule 1 results in Follow(*stmt*) = {**;**} and Follow(*stmt-seq'*) = {$}. Rule 2 has no effect. A second pass results in no further changes. Thus, we have computed the Follow sets

Follow(*stmt-sequence*) = {$}
Follow(*stmt*) = {**;**}
Follow(*stmt-seq'*) = {$} §

4.3.3 Constructing LL(1) Parsing Tables

Consider now the original construction of the entries of the LL(1) parsing table, as given in Section 4.2.2:

1. If $A \rightarrow \alpha$ is a production choice, and there is a derivation $\alpha \Rightarrow^* a\ \beta$, where a is a token, then add $A \rightarrow \alpha$ to the table entry $M[A, a]$.
2. If $A \rightarrow \alpha$ is a production choice and there are derivations $\alpha \Rightarrow^* \varepsilon$ and $S\ \$ \Rightarrow^* \beta\ A\ a\ \gamma$, where S is the start symbol and a is a token (or \$), then add $A \rightarrow \alpha$ to the table entry $M[A,a]$.

Clearly, the token a in rule 1 is in First(α), and the token a of rule 2 is in Follow(A). Thus, we have arrived at the following algorithmic construction of the LL(1) parsing table:

Construction of the LL(1) parsing table $M[N, T]$

Repeat the following two steps for each nonterminal A and production choice $A \rightarrow \alpha$.

1. For each token a in First(α), add $A \rightarrow \alpha$ to the entry $M[A, a]$.
2. If ε is in First(α), for each element a of Follow(A) (a token or \$), add $A \rightarrow \alpha$ to $M[A, a]$.

The following theorem is essentially a direct consequence of the definition of an LL(1) grammar and the parsing table construction just given, and we leave its proof to the exercises:

Theorem

A grammar in BNF is **LL(1)** if the following conditions are satisfied.

1. For every production $A \rightarrow \alpha_1 \mid \alpha_2 \mid \ldots \mid \alpha_n$, First($\alpha_i$) $\cap$ First(α_j) is empty for all i and j, $1 \leq i, j \leq n$, $i \neq j$.
2. For every nonterminal A such that First(A) contains ε, First(A) $\cap$ Follow(A) is empty.

We now look at some examples of parsing tables for grammars we have previously seen.

Example 4.15

Consider the simple expression grammar we have been using as a standard example throughout this chapter. This grammar as originally given (see Example 4.9 earlier) is left recursive. In the last section we wrote out an equivalent grammar using left recursion removal, as follows:

exp → *term exp'*
exp' → *addop term exp'* | ε
addop → + | -
term → *factor term'*
term' → *mulop factor term'* | ε
mulop → *
factor → (*exp*) | **number**

We must compute First and Follow sets for the nonterminals for this grammar. We leave this computation for the exercises and simply state the result:

First(*exp*) = { (, **number** }	Follow(*exp*) = {$,) }
First(*exp'*) = {+, -, ε}	Follow(*exp'*) = {$,) }
First(*addop*) = {+, -}	Follow(*addop*) = { (, **number** }
First(*term*) = { (, **number** }	Follow(*term*) = {$,), +, - }
First(*term'*) = {*, ε}	Follow(*term'*) = {$,), +, - }
First(*mulop*) = {*}	Follow(*mulop*) = { (, **number** }
First(*factor*) = { (, **number** }	Follow(*factor*) = {$,), +, -, * }

Applying the LL(1) parsing table construction just described, we get the table as given in Table 4.4, page 163. §

Example 4.16 Consider the simplified grammar of if-statements

statement → *if-stmt* | **other**
if-stmt → **if** (*exp*) *statement else-part*
else-part → **else** *statement* | ε
exp → **0** | **1**

The First sets of this grammar were computed in Example 4.10 and the Follow sets were computed in Example 4.13. We list these sets again here:

First(*statement*) = {**if**, **other**}	Follow(*statement*) = {$, **else**}
First(*if-stmt*) = {**if**}	Follow(*if-stmt*) = {$, **else**}
First(*else-part*) = {**else**, ε}	Follow(*else-part*) = {$, **else**}
First(*exp*) = {**0**, **1**}	Follow(*exp*) = {)}

Constructing the LL(1) parsing table gives Table 4.3, page 156. §

Example 4.17 Consider the grammar of Example 4.4 (with left factoring applied):

stmt-sequence → *stmt stmt-seq'*
stmt-seq' → **;** *stmt-sequence* | ε
stmt → **s**

This grammar has the following First and Follow sets

First(*stmt-sequence*) = {**s**}	Follow(*stmt-sequence*) = {$}
First(*stmt*) = {**s**}	Follow(*stmt*) = {**;**, $}
First(*stmt-seq'*) = {**;**, ε}	Follow(*stmt-seq'*) = {$}

and the LL(1) parsing table at the top of the next page.

M[N, T]	s	;	$
stmt-sequence	*stmt-sequence* → *stmt stmt-seq'*		
stmt	*stmt* → s		
stmt-seq'		*stmt-seq'* → ; *stmt-sequence*	*stmt-seq'* → ε

§

4.3.4 Extending the Lookahead: LL(*k*) Parsers

The previous work can be extended to k symbols of lookahead. For example, we can define $\text{First}_k(\alpha) = \{ w_k \mid \alpha \Rightarrow^* w \}$, where w is a string of tokens and w_k = the first k tokens of w (or w, if w has fewer than k tokens). Similarly, we can define $\text{Follow}_k(A) = \{ w_k \mid S\$ \Rightarrow^* \alpha A w \}$. While these definitions are less "algorithmic" than our definitions for the case $k = 1$, algorithms to compute these sets can be developed, and the construction of the LL(k) parsing table can be performed as before.

Several complications occur in LL(k) parsing, however. First, the parsing table becomes much larger, since the number of columns increases exponentially with k. (To a certain extent this can be countered by using table compression methods.) Second, the parsing table itself does not express the complete power of LL(k) parsing, essentially because the Follow strings do not all occur in all contexts. Thus, parsing using the table as we have constructed it is distinguished from LL(k) parsing by calling it **Strong LL(k) parsing, or SLL(k) parsing**. We refer the reader to the Notes and References section for more information.

LL(k) and SLL(k) parsers are uncommon, partially because of the added complexity, but primarily because of the fact that a grammar that fails to be LL(1) is in practice likely not to be LL(k) for any k. For instance, a grammar with left recursion is never LL(k), no matter how large the k. Recursive-descent parsers, however, are able to selectively use larger lookaheads when needed and even, as we have seen, use ad hoc methods to parse grammars that are not LL(k) for any k.

4.4 A RECURSIVE-DESCENT PARSER FOR THE TINY LANGUAGE

In this section we discuss the complete recursive-descent parser for the TINY language listed in Appendix B. The parser constructs a syntax tree as described in Section 3.7 of the previous chapter and, additionally, prints a representation of the syntax tree to the listing file. The parser uses the EBNF as given in Figure 4.9, corresponding to the BNF of Chapter 3, Figure 3.6.

Figure 4.9
Grammar of the TINY language in EBNF

program → *stmt-sequence*
stmt-sequence → *statement* { **;** *statement* }
statement → *if-stmt* | *repeat-stmt* | *assign-stmt* | *read-stmt* | *write-stmt*
if-stmt → **if** *exp* **then** *stmt-sequence* [**else** *stmt-sequence*] **end**
repeat-stmt → **repeat** *stmt-sequence* **until** *exp*
assign-stmt → **identifier** **:=** *exp*
read-stmt → **read** ***identifier***
write-stmt → **write** *exp*
exp → *simple-exp* [*comparison-op simple-exp*]
comparison-op → **<** | **=**
simple-exp → *term* { *addop term* }
addop → **+** | **-**
term → *factor* { *mulop factor* }
mulop → ***** | **/**
factor → **(** *exp* **)** | ***number*** | ***identifier***

The TINY parser follows closely the outline of recursive descent parsing given in Section 4.1. The parser consists of two code files, **parse.h** and **parse.c**. The **parse.h** file (Appendix B, lines 850–865) is extremely simple: it consists of a single declaration

```
TreeNode * parse(void);
```

defining the main parse routine **parse** that returns a pointer to the syntax tree constructed by the parser. The **parse.c** file is given in Appendix B, lines 900–1114. It consists of 11 mutually recursive procedures that correspond directly to the EBNF grammar of Figure 4.9: one for *stmt-sequence*, one for *statement*, one each for the five different kinds of statements, and four for the different precedence levels of expressions. Operator nonterminals are not included as procedures, but are recognized as part of their associated expressions. There is also no procedure corresponding to *program*, since a program is just a statement sequence, so the **parse** routine simply calls **stmt_sequence**.

The parser code also includes a static variable **token** that keeps the lookahead token, a **match** procedure that looks for a specific token, calling **getToken** if it finds it and declaring error otherwise, and a **syntaxError** procedure that prints an error message to the listing file. The main **parse** procedure initializes **token** to the first token in the input, calls **stmt_sequence**, and then checks for the end of the source file before returning the tree constructed by **stmt_sequence**. (If there are more tokens after **stmt_sequence** returns, this is an error.)

The contents of each recursive procedure should be relatively self-explanatory, with the possible exception of **stmt_sequence**, which has been written in a somewhat more complex form to improve error handling; this will be explained in the discussion of error handling given shortly. The recursive parsing procedures make use of three utility procedures, which are gathered for simplicity into the file **util.c** (Appendix

B, lines 350–526), with interface **util.h** (Appendix B, lines 300–335). These procedures are

1. **newStmtNode** (lines 405–421), which takes a parameter indicating the kind of statement, and allocates a new statement node of that kind, returning a pointer to the newly allocated node;
2. **newExpNode** (lines 423–440), which takes a parameter indicating the kind of expression and allocates a new expression node of that kind, returning a pointer to the newly allocated node; and
3. **copyString** (lines 442–455), which takes a string parameter, allocates sufficient space for a copy, and copies the string, returning a pointer to the newly allocated copy.

The **copyString** procedure is made necessary by the fact that the C language does not automatically allocate space for strings and by the fact that the scanner reuses the same space for the string values (or lexemes) of the tokens it recognizes.

A procedure **printTree** (lines 473–506) is also included in **util.c** that writes a linear version of the syntax tree to the listing, so that we may view the result of a parse. This procedure is called from the main program, under the control of the global **traceParse** variable.

The **printTree** procedure operates by printing node information and then indenting to print child information. The actual tree can be reconstructed from this indentation. The syntax tree of the sample TINY program (see Chapter 3, Figures 3.8 and 3.9, pp. 137–138) is printed to the listing file by **traceParse** as shown in Figure 4.10.

Figure 4.10
Display of a TINY syntax tree by the **printTree** procedure

```
Read: x
If
  Op: <
    const: 0
    Id: x
  Assign to: fact
    const: 1
  Repeat
    Assign to: fact
      Op: *
        Id: fact
        Id: x
    Assign to: x
      Op: -
        Id: x
        const: 1
    Op: =
      Id: x
      const: 0
  Write
    Id: fact
```

4.5 ERROR RECOVERY IN TOP-DOWN PARSERS

The response of a parser to syntax errors is often a critical factor in the usefulness of a compiler. Minimally, a parser must determine whether a program is syntactically correct or not. A parser that performs this task alone is called a **recognizer**, since all it does is recognize strings in the context-free language generated by the grammar of the programming language in question. It is worth stating that at a minimum, any parser must behave like a recognizer—that is, if a program contains a syntax error, the parser must indicate that *some* error exists, and conversely, if a program contains no syntax errors, then the parser should not claim that an error exists.

Beyond this minimal behavior, a parser can exhibit many different levels of responses to errors. Usually, a parser will attempt to give a meaningful error message, at least for the first error encountered, and also attempt to determine as closely as possible the location where that error has occurred. Some parsers may go so far as to attempt some form of **error correction** (or, perhaps more appropriately, **error repair**), where the parser attempts to infer a correct program from the incorrect one given. If this is attempted, most of the time it is limited to easy cases, such as missing punctuation. There exists a body of algorithms that can be applied to find a correct program that is closest in some sense to the one given (usually in terms of number of tokens that must be inserted, deleted, or changed). Such **minimal distance error correction** is usually too inefficient to be applied to every error and, in any case, results in error repair that is often very far from what the programmer actually intended. Thus, it is seldom seen in actual parsers. Compiler writers find it difficult enough to generate meaningful error messages without trying to do error correction.

Most of the techniques for error recovery are ad hoc, in the sense that they apply to specific languages and specific parsing algorithms, with many special cases for individual situations. General principles are hard to come by. Some important considerations that apply are the following.

1. A parser should try to determine that an error has occurred *as soon as possible.* Waiting too long before declaring error means the location of the actual error may have been lost.
2. After an error has occurred, the parser must pick a likely place to resume the parse. A parser should always try to parse as much of the code as possible, in order to find as many real errors as possible during a single translation.
3. A parser should try to avoid the **error cascade problem**, in which one error generates a lengthy sequence of spurious error messages.
4. A parser must avoid infinite loops on errors, in which an unending cascade of error messages is generated without consuming any input.

Some of these goals conflict with each other, so that a compiler writer is forced to make trade-offs during the construction of an error handler. For example, avoiding the error cascade and infinite loop problems can cause the parser to skip some of the input, compromising the goal of processing as much of the input as possible.

4.5.1 Error Recovery in Recursive-Descent Parsers

A standard form of error recovery in recursive-descent parsers is called **panic mode**. The name derives from the fact that, in complex situations, the error handler will con-

sume a possibly large number of tokens in an attempt to find a place to resume parsing (in the worst case, it might even consume the entire rest of the program, in which case it is no better than simply exiting after the error). However, when implemented with some care, this can be a much better method for error recovery than its name implies.[4] Panic mode has the added attraction that it virtually ensures that the parser cannot get into an infinite loop during error recovery.

The basic mechanism of panic mode is to provide each recursive procedure with an extra parameter consisting of a set of **synchronizing tokens**. As parsing proceeds, tokens that may function as synchronizing tokens are added to this set as each call occurs. If an error is encountered, the parser **scans ahead**, throwing away tokens until one of the synchronizing set of tokens is seen in the input, whence parsing is resumed. Error cascades are avoided (to a certain extent) by not generating new error messages while this forward scan takes place.

The important decisions to be made in this error recovery method are what tokens to add to the synchronizing set at each point in the parse. Generally, Follow sets are important candidates for such synchronizing tokens. First sets may also be used to prevent the error handler from skipping important tokens that begin major new constructs (like statements or expressions). First sets are also important, in that they allow a recursive descent parser to detect errors early in the parse, which is always helpful in any error recovery. It is important to realize that panic mode works best when the compiler "knows" when *not* to panic. For example, missing punctuation symbols such as semicolons or commas, and even missing right parentheses, should not always cause an error handler to consume tokens. Of course, care must be taken to ensure that an infinite loop cannot occur.

We illustrate panic mode error recovery by sketching in pseudocode its implementation in the recursive descent calculator of Section 4.1.2 (see also Figure 4.1). In addition to the *match* and *error* procedures, which remain essentially the same (except that *error* no longer exits immediately), we have two more procedures, *checkinput*, which performs the early lookahead checking, and *scanto*, which is the panic mode token consumer proper:

```
procedure scanto ( synchset ) ;
begin
  while not ( token in synchset ∪ { $ }) do
    getToken ;
end scanto ;

procedure checkinput ( firstset, followset ) ;
begin
  if not ( token in firstset ) then
    error ;
    scanto ( firstset ∪ followset ) ;
  end if ;
end;
```

Here the $ refers to the end of input (EOF).

4. Wirth [1976], in fact, calls panic mode the "don't panic" rule, presumably in an attempt to improve its image.

These procedures are used as follows in the *exp* and *factor* procedures (which now take a *synchset* parameter):

```
procedure exp ( synchset ) ;
begin
  checkinput ( { (, number }, synchset ) ;
  if not ( token in synchset ) then
    term ( synchset ) ;
    while token = + or token = - do
      match (token) ;
      term ( synchset ) ;
    end while ;
    checkinput ( synchset, { (, number }) ;
 end if;
end exp ;

procedure factor ( synchset ) ;
begin
 checkinput ( { (, number }, synchset ) ;
 if not ( token in synchset ) then
  case token of
  ( : match( ( ) ;
      exp ( { ) } ) ;
      match( ) ) ;
  number :
      match(number) ;
  else error ;
  end case ;
  checkinput ( synchset, { (, number }) ;
 end if ;
end factor ;
```

Note how *checkinput* is called twice in each procedure: once to verify that a token in the First set is the next token in the input and a second time to verify that a token in the Follow set (or *synchset*) is the next token on exit.

This form of panic mode will generate reasonable errors (useful error messages can also be added as parameters to *checkinput* and *error*). For example, the input string `(2+-3)*4-+5` will generate exactly two error messages (one at the first minus sign and one at the second plus sign).

We note that, in general, *synchset* is to be passed down in the recursive calls, with new synchronizing tokens added as appropriate. In the case of *factor*, an exception is made after a left parenthesis is seen: *exp* is called with right parenthesis only as its follow set (*synchset* is discarded). This is typical of the kind of ad hoc analysis that accompanies panic mode error recovery. (We did this so that, for example, the expression `(2+*)` would not generate a spurious error message at the right parenthesis.) We leave an analysis of the behavior of this code, and its implementation in C, to the exercises. Unfortunately, to obtain the best error messages and error recovery, virtually every token test must be examined for the possibility that a more general test, or an earlier test, may improve error behavior.

4.5.2 Error Recovery in LL(1) Parsers

Panic mode error recovery can be implemented in LL(1) parsers in a similar manner to the way it is implemented in recursive-descent parsing. Since the algorithm is nonrecursive, a new stack is required to keep the *synchset* parameters, and a call to *checkinput* must be scheduled by the algorithm before each generate action of the algorithm (when a nonterminal is at the top of the stack).[5] Note that the primary error situation occurs with a nonterminal A at the top of the stack and the current input token is not in First(A) (or Follow(A), if ε is in First(A)). The case where a token is at the top of the stack, and it is not the same as the current input token, does not normally occur, since tokens are, in general, only pushed onto the stack when they are actually seen in the input (table compression methods may compromise this slightly). We leave the modifications to the parsing algorithm of Figure 4.2, page 155, to the exercises.

An alternative to the use of an extra stack is to statically build the sets of synchronizing tokens directly into the LL(1) parsing table, together with the corresponding actions that *checkinput* would take. Given a nonterminal A at the top of the stack and an input token that is not in First(A) (or Follow(A), if ε is in First(A)), there are three possible alternatives:

1. Pop A from the stack.
2. Successively pop tokens from the input until a token is seen for which we can restart the parse.
3. Push a new nonterminal onto the stack.

We choose alternative 1 if the current input token is \$ or is in Follow(A) and alternative 2 if the current input token is not \$ and is not in First(A) $\cup$ Follow(A). Option 3 is occasionally useful in special situations, but is rarely appropriate (we shall discuss one case shortly). We indicate the first action in the parsing table by the notation *pop* and the second by the notation *scan*. (Note that a *pop* action is equivalent to a reduction by an ε-production.)

With these conventions, the LL(1) parsing table (Table 4.4) looks as in Table 4.9. The behavior of an LL(1) parser using this table, given the string **(2+*)**, is shown in Table 4.10. In that table, the parse is shown only from the first error on (so the prefix **(2+** has already been successfully matched). We also use the abbreviations E for *exp*, E' for *exp'*, and so on. Note that there are two adjacent error moves before the parse resumes successfully. We can arrange to suppress an error message on the second error move by requiring, after the first error, that the parser make one or more successful moves before generating any new error messages. Thus, error message cascades would be avoided.

There is (at least) one problem in this error recovery method that calls for special action. Since many error actions pop a nonterminal from the stack, it is possible for the stack to become empty, with some input still to parse. A simple case of this in the example just given is any string beginning with a right parenthesis: this will cause E to be immediately popped, leaving the stack empty with all the input yet to be consumed.

---

5. Calls to *checkinput* at the end of a match, as in the recursive-descent code, can also be scheduled by a special stack symbol in a similar fashion to the value computation scheme of Section 4.2.4, page 167.

One possible action that the parser can take in this situation is to push the start symbol onto the stack and scan forward in the input until a symbol in the First set of the start symbol is seen.

Table 4.9
LL(1) parsing table (Table 4.4) with error recovery entries

M[N, T]	(	number	)	+	-	*	$
exp	*exp* → *term exp'*	*exp* → *term exp'*	pop	scan	scan	scan	pop
exp'	scan	scan	*exp'* → ε	*exp'* → *addop term exp'*	*exp'* → *addop term exp'*	scan	*exp'* → ε
addop	pop	pop	scan	*addop* → +	*addop* → -	scan	pop
term	*term* → *factor term'*	*term* → *factor term'*	pop	pop	pop	scan	pop
term'	scan	scan	*term'* → ε	*term'* → ε	*term'* → ε	*term'* → *mulop factor term'*	*term'* → ε
mulop	pop	pop	scan	scan	scan	*mulop* → *	pop
factor	*factor* → (*exp*)	*factor* → number	pop	pop	pop	pop	pop

Table 4.10
Moves of an LL(1) parser using Table 4.9

Parsing stack	Input	Action
$ *E' T'*) *E' T*	*) $	scan (error)
$ *E' T'*) *E' T*	) $	pop (error)
$ *E' T'*) *E'*	) $	*E'* → ε
$ *E' T'*)	) $	match
$ *E' T'*	$	*T'* → ε
$ *E'*	$	*E'* → ε
$	$	accept

4.5.3 Error Recovery in the TINY Parser

The error handling of the TINY parser, as given in Appendix B, is extremely rudimentary: only a very primitive form of panic mode recovery is implemented, without the synchronizing sets. The `match` procedure simply declares error, stating which token it found that it did not expect. In addition, the procedures `statement` and `factor` declare error when no correct choice is found. Also the `parse` procedure declares error

if a token other than end of file is found after the parse finishes. The principal error message generated is "unexpected token," which can be very unhelpful to the user. In addition, the parser makes no attempt to avoid error cascades. For example, the sample program with a semicolon added after the write-statement

```
...
 5: read x ;
 6: if 0 < x then
 7:   fact := 1;
 8:   repeat
 9:     fact := fact * x;
10:     x := x - 1
11:   until x = 0;
12:   write fact; {<- - BAD SEMICOLON! }
13: end
14:
```

causes the following *two* error messages to be generated (when only one error has occurred):

```
Syntax error at line 13: unexpected token -> reserved word: end
Syntax error at line 14: unexpected token -> EOF
```

And the same program with the comparison < deleted in the second line of code

```
...
 5: read x ;
 6: if 0 x then { <- - COMPARISON MISSING HERE! }
 7:   fact := 1;
 8:   repeat
 9:     fact := fact * x;
10:     x := x - 1
11:   until x = 0;
12:   write fact
13: end
14:
```

causes *four* error messages to be printed in the listing:

```
Syntax error at line 6: unexpected token -> ID, name = x
Syntax error at line 6: unexpected token -> reserved word: then
Syntax error at line 6: unexpected token -> reserved word: then
Syntax error at line 7: unexpected token -> ID, name = fact
```

On the other hand, some of the TINY parser's behavior is reasonable. For example, a missing (rather than an extra) semicolon will generate only one error message, and the parser will go on to build the correct syntax tree as if the semicolon had been there all along, thus performing a rudimentary form of error correction in this one case. This behavior results from two coding facts. The first is that the **match** procedure does not

consume a token, which results in behavior that is identical to inserting a missing token. The second is that the **stmt_sequence** procedure has been written so as to connect up as much of the syntax tree as possible in the case of an error. In particular, care was taken to make sure that the sibling pointers are connected up whenever a non-nil pointer is found (the parser procedures are designed to return a nil syntax tree pointer if an error is found). Also, the obvious way of writing the body of **stmt_sequence** based on the EBNF

```
statement();
while (token==SEMI)
{ match(SEMI);
  statement();
}
```

is instead written with a more complicated loop test:

```
statement();
while ((token!=ENDFILE) && (token!=END) &&
       (token!=ELSE) && (token!=UNTIL))
{ match(SEMI);
  statement();
}
```

The reader may note that the four tokens in this negative test comprise the Follow set for *stmt-sequence*. This is not an accident, as a test may either look for a token in the First set (as do the procedures for *statement* and *factor*) or look for a token *not* in the Follow set. This latter is particularly effective in error recovery, since if a First symbol is missing, the parse would stop. We leave to the exercises a trace of the program behavior to show that a missing semicolon would indeed cause the rest of the program to be skipped if **stmt_sequence** were written in the first form given.

Finally, we note that the parser has also been written in such a way that it cannot get into an infinite loop when it encounters errors (the reader should have worried about this when we noted that **match** does not consume an unexpected token). This is because, in an arbitrary path through the parsing procedures, eventually either the default case of **statement** or **factor** must be encountered, and both of these *do* consume a token while generating an error message.

EXERCISES

4.1 Write pseudocode for *term* and *factor* corresponding to the pseudocode for *exp* in Section 4.1.2 (page 150) that constructs a syntax tree for simple arithmetic expressions.

4.2 Given the grammar $A \rightarrow (\ A\ )\ A \mid \varepsilon$, write pseudocode to parse this grammar by recursive-descent.

4.3 Given the grammar

statement → *assign-stmt* | *call-stmt* | **other**
assign-stmt → **identifier** := *exp*
call-stmt → **identifier** (*exp-list*)

write pseudocode to parse this grammar by recursive-descent.

4.4 Given the grammar

lexp → **number** | **(** *op lexp-seq* **)**
op → **+** | **-** | *****
lexp-seq → *lexp-seq lexp* | *lexp*

write pseudocode to compute the numeric value of an *lexp* by recursive-descent (see Exercise 3.13 of Chapter 3).

4.5 Show the actions of an LL(1) parser that uses Table 4.4 (page 163) to recognize the following arithmetic expressions:

a. `3+4*5-6` **b.** `3*(4-5+6)` **c.** `3-(4+5*6)`

4.6 Show the actions of an LL(1) parser that uses the table of Section 4.2.2, page 155, to recognize the following strings of balanced parentheses:

a. `(())()` **b.** `(()())` **c.** `()(())`

4.7 Given the grammar $A \rightarrow (\ A\ )\ A \mid \varepsilon$,

a. Construct First and Follow sets for the nonterminal A.

b. Show this grammar is LL(1).

4.8 Consider the grammar

lexp → *atom* | *list*
atom → **number** | **identifier**
list → **(** *lexp-seq* **)**
lexp-seq → *lexp-seq lexp* | *lexp*

a. Remove the left recursion.

b. Construct First and Follow sets for the nonterminals of the resulting grammar.

c. Show that the resulting grammar is LL(1).

d. Construct the LL(1) parsing table for the resulting grammar.

e. Show the actions of the corresponding LL(1) parser, given the input string `(a (b (2)) (c))`.

4.9 Consider the following grammar (similar, but not identical to the grammar of Exercise 4.8):

lexp → *atom* | *list*
atom → **number** | **identifier**
list → **(** *lexp-seq* **)**
lexp-seq → *lexp* **,** *lexp-seq* | *lexp*

a. Left factor this grammar.

b. Construct First and Follow sets for the nonterminals of the resulting grammar.

c. Show that the resulting grammar is LL(1).

d. Construct the LL(1) parsing table for the resulting grammar.

e. Show the actions of the corresponding LL(1) parser, given the input string `(a, (b, (2)), (c))`.

4.10 Consider the following grammar of simplified C declarations:

declaration → *type var-list*
type → **int** | **float**
var-list → **identifier**, *var-list* | **identifier**

a. Left factor this grammar.
b. Construct First and Follow sets for the nonterminals of the resulting grammar.
c. Show that the resulting grammar is LL(1).
d. Construct the LL(1) parsing table for the resulting grammar.
e. Show the actions of the corresponding LL(1) parser, given the input string **int x,y,z**.

4.11 An LL(1) parsing table such as that of Table 4.4 (page 163) generally has many blank entries representing errors. In many cases, all the blank entries in a row can be replaced by a single **default entry**, thus decreasing the size of the table considerably. Potential default entries occur in nonterminal rows when a nonterminal has a single production choice or when it has an ε-production. Apply these ideas to Table 4.4. What are the drawbacks of such a scheme, if any?

4.12 **a.** Can an LL(1) grammar be ambiguous? Why or why not?
b. Can an ambiguous grammar be LL(1)? Why or why not?
c. Must an unambiguous grammar be LL(1)? Why or why not?

4.13 Show that a left-recursive grammar cannot be LL(1).

4.14 Prove the theorem on page 178 that concludes a grammar is LL(1) from two conditions on its First and Follow sets.

4.15 Define the operator $\oplus$ on two sets of token strings S_1 and S_2 as follows: $S_1 \oplus S_2 = \{ \text{First}(xy) \mid x \in S_1, y \in S_2 \}$.
a. Show that, for any two nonterminals A and B, $\text{First}(AB) = \text{First}(A) \oplus \text{First}(B)$.
b. Show that the two conditions of the theorem on page 178 can be replaced by the single condition: if $A \to \alpha$ and $A \to \beta$, then $(\text{First}(\alpha) \oplus \text{Follow}(A)) \cap (\text{First}(\beta) \oplus \text{Follow}(A))$ is empty.

4.16 A nonterminal A is **useless** if there is no derivation from the start symbol to a string of tokens in which A appears.
a. Give a mathematical formulation of this property.
b. Is it likely that a programming language grammar will have a useless symbol? Explain.
c. Show that, if a grammar has a useless symbol, the computation of First and Follow sets as given in this chapter may produce sets that are too large to accurately construct an LL(1) parsing table.

4.17 Give an algorithm that removes useless nonterminals (and associated productions) from a grammar without changing the language recognized. (See the previous exercise.)

4.18 Show that the converse of the theorem on page 178 is true, if a grammar has no useless nonterminals (see Exercise 4.16).

4.19 Give details of the computation of the First and Follow sets listed in Example 4.15 (page 178).

4.20 **a.** Construct the First and Follow sets for the nonterminals of the left-factored grammar of Example 4.7 (page 165).
b. Construct the LL(1) parsing table using your results of part (a).

4.21 Given the grammar $A \rightarrow a\ A\ a \mid \varepsilon$,

a. Show that this grammar is not LL(1).

b. An attempt to write a recursive-descent parser for this grammar is represented by the following pseudocode.

```
procedure A ;
begin
  if token = a then
    getToken ;
    A ;
    if token = a then getToken ;
    else error ;
  else if token <> $ then error ;
end A ;
```

Show that this procedure will not work correctly.

c. A **backtracking** recursive-descent parser for this language *can* be written but requires the use of an *unGetToken* procedure that takes a token as a parameter and returns that token to the front of the stream of input tokens. It also requires that the procedure *A* be written as a Boolean function that returns success or failure, so that when *A* calls itself, it can test for success before consuming another token, and so that, if the $A \rightarrow a\ A\ a$ choice fails, the code can go on to try the $A \rightarrow \varepsilon$ alternative. Rewrite the pseudocode of part (b) according to this recipe, and trace its operation on the string *aaaa*$.

4.22 In the TINY grammar of Figure 4.9 (page 181), a clear distinction is not made between Boolean and arithmetic expressions. For instance, the following is a syntactically legal TINY program:

```
if 0 then write 1>0 else x := (x<1)+1 end
```

Rewrite the TINY grammar so that only Boolean expressions (expressions containing comparison operators) are allowed as the test of an if- or repeat-statement, and only arithmetic expressions are allowed in write- or assign-statements or as operands of any of the operators.

4.23 Add the Boolean operators **and**, **or**, and **not** to the TINY grammar of Figure 4.9 (page 181). Give them the properties described in Exercise 3.5 (page 139), as well as lower precedence than all arithmetic operators. Make sure any expression can be either Boolean or integer.

4.24 The changes to the TINY syntax in Exercise 4.23 have made the problem described in Exercise 4.22 worse. Rewrite the answer to Exercise 4.23 to rigidly distinguish between Boolean and arithmetic expressions, and incorporate this into the solution to Exercise 4.22.

4.25 The panic mode error recovery for the simple arithmetic expression grammar, as described in Section 4.5.1, still suffers from some drawbacks. One is that the while-loops that test for operators should continue to operate under certain circumstances. For instance, the expression `(2)(3)` is missing its operator in between the factors, but the error handler consumes the second factor without restarting the parse. Rewrite the pseudocode to improve its behavior in this case.

4.26 Trace the actions of an LL(1) parser on the input `(2+-3)*4-+5`, using the error recovery as given in Table 4.9 (page 187).

4.27 Rewrite the LL(1) parsing algorithm of Figure 4.2 (page 155) to implement full panic mode error recovery, and trace its behavior on the input `(2+-3)*4-+5`.

4.28 **a.** Trace the operation of the **`stmt_sequence`** procedure in the TINY parser to verify that the correct syntax tree is constructed for the following TINY program, despite the missing semicolon:

```
x := 2
y := x + 2
```

b. What syntax tree is constructed for the following (incorrect) program:

```
x 2
y := x + 2
```

c. Suppose that the **`stmt_sequence`** procedure was written instead using the simpler code sketched on page 189:

```
statement();
while (token==SEMI)
{ match(SEMI);
  statement();
}
```

What syntax trees are constructed for the programs in parts (a) and (b) using this version of the **`stmt_sequence`** procedure?

PROGRAMMING EXERCISES

4.29 Add the following to the simple integer arithmetic recursive-descent calculator of Figure 4.1, pages 148–149 (make sure they have the correct associativity and precedence):

a. Integer division with the symbol `/`.

b. Integer mod with the symbol `%`.

c. Integer exponentiation with the symbol `^`. (Warning: This operator has higher precedence than multiplication and is right associative.)

d. Unary minus with the symbol `-`. (See Exercise 3.12, page 140.)

4.30 Rewrite the recursive-descent calculator of Figure 4.1 (pages 148–149) so that it computes with floating-point numbers instead of integers.

4.31 Rewrite the recursive-descent calculator of Figure 4.1 so that it *distinguishes* between floating-point and integer values, rather than simply computing everything as integers or floating-point numbers. (Hint: A "value" is now a record with a flag indicating whether it is integer or floating point.)

4.32 **a.** Rewrite the recursive-descent calculator of Figure 4.1 so that it returns a syntax tree according to the declarations of Section 3.3.2 (page 111).

b. Write a function that takes as a parameter the syntax tree produced by your code of part (a) and returns the calculated value by traversing the tree.

4.33 Write a recursive-descent calculator for simple integer arithmetic similar to that of Figure 4.1, but use the grammar of Figure 4.4 (page 160).

4.34 Consider the following grammar:

$$lexp \rightarrow \texttt{number} \mid \texttt{(}\ op\ lexp\text{-}seq\ \texttt{)}$$
$$op \rightarrow \texttt{+} \mid \texttt{-} \mid \texttt{*}$$
$$lexp\text{-}seq \rightarrow lexp\text{-}seq\ lexp \mid lexp$$

This grammar can be thought of as representing simple integer arithmetic expressions in LISP-like prefix form. For example, the expression `34-3*42` would be written in this grammar as `(- 34 (* 3 42))`.

Write a recursive-descent calculator for the expressions given by this grammar.

4.35 **a.** Devise a syntax tree structure for the grammar of the previous exercise and write a recursive-descent parser for it that returns a syntax tree.

b. Write a function that takes as a parameter the syntax tree produced by your code of part (a) and returns the calculated value by traversing the tree.

4.36 **a.** Use the grammar for regular expressions of Exercise 3.4, page 138 (appropriately disambiguated) to construct a recursive-descent parser that reads a regular expression and performs Thompson's construction of an NFA (see Chapter 2).

b. Write a procedure that takes an NFA data structure as produced by the parser of part (a) and constructs an equivalent DFA according to the subset construction.

c. Write a procedure that takes the data structure as produced by your procedure of part (b) and finds longest substrings matches in a text file according to the DFA it represents. (Your program has now become a "compiled" version of grep!)

4.37 Add the comparison operators `<=` (less than or equal to), `>` (greater than), `>=` (greater than or equal to), and `<>` (not equal to) to the TINY parser. (This will require adding these tokens and changing the scanner as well, but should not require a change to the syntax tree.)

4.38 Incorporate your grammar changes from Exercise 4.22 into the TINY parser.

4.39 Incorporate your grammar changes from Exercise 4.23 into the TINY parser.

4.40 **a.** Rewrite the recursive-descent calculator program of Figure 4.1 (pages 148–149) to implement panic mode error recovery as outlined in Section 4.5.1.

b. Add useful error messages to your error handling of part (a).

4.41 One of the reasons that the TINY parser produces poor error messages is that the `match` procedure is limited to printing only the current token when an error occurs, rather than printing both the current token and the expected token, and that no special error message is passed to the `match` procedure from its call sites. Rewrite the `match` procedure so that it prints the expected token as well as the current token and also passes along an error message to the `syntaxError` procedure. This will require a rewrite of the `syntaxError` procedure, as well as changes to the calls of `match` to include an appropriate error message.

4.42 Add synchronizing sets of tokens and panic mode error recovery to the TINY parser, as described on page 184.

4.43 (From Wirth [1976]) Data structures based on syntax diagrams can be used by a

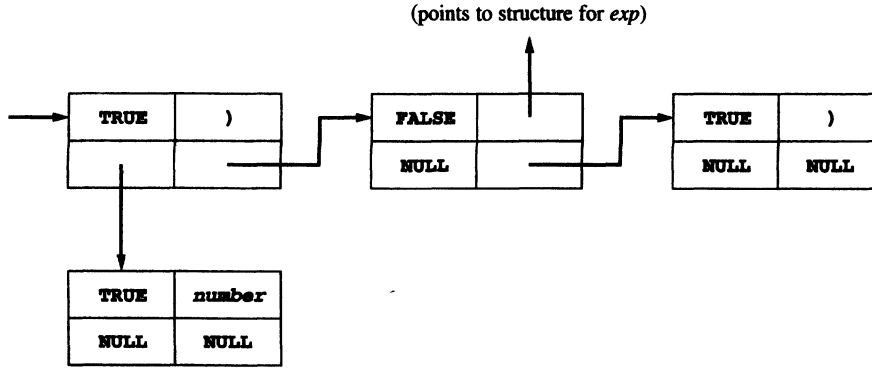
"generic" recursive-descent parser that will parse any set of LL(1) grammar rules. A suitable data structure is given by the following C declarations:

```
typedef struct rulerec
     { struct rulerec *next,*other;
       int isToken;
       union
       { Token name;
         struct rulerec *rule;
       } attr;
     } Rulerec;
```

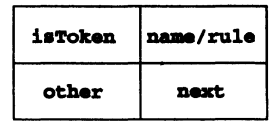
The **next** field is used to point to the next item in the grammar rule, and the **other** field is used to point to alternatives given by the | metasymbol. Thus, the data structure for the grammar rule

$$factor \rightarrow \texttt{(}\ exp\ \texttt{)} \mid \textbf{number}$$

would look as follows



where the fields of the record structure are shown as follows



a. Draw the data structures for the rest of the grammar rules in Figure 4.4, page 160 (Hint: You will need a special token to represent ε internally in the data structure.)

b. Write a generic parse procedure that uses these data structures to recognize an input string.

c. Write a parser generator that reads BNF rules (either from a file or standard input) and generates the preceding data structures.

NOTES AND REFERENCES

Recursive-descent parsing has been a standard method for parser construction since the early 1960s and the introduction of BNF rules in the Algol60 report [Naur, 1963]. For an early description of the method, see Hoare [1962]. Backtracking recursive-descent parsers have recently become popular in strongly typed lazy functional languages such as Haskell and Miranda, where this form of recursive-descent parsing is called combinator parsing. For a description of this method see Peyton Jones and Lester [1992] or Hutton [1992]. The use of EBNF in conjunction with recursive-descent parsing has been popularized by Wirth [1976].

LL(1) parsing was studied extensively in the 1960s and early 1970s. For an early description see Lewis and Stearns [1968]. A survey of LL(k) parsing may be found in Fischer and LeBlanc [1991], where an example of an LL(2) grammar that is not SLL(2) can be found. For practical uses of LL(k) parsing, see Parr, Dietz, and Cohen [1992].

There are, of course, many more top-down parsing methods than just the two we have studied in this chapter. For an example of another, more general, method, see Graham, Harrison, and Ruzzo [1980].

Panic mode error recovery is studied in Wirth [1976] and Stirling [1985]. Error recovery in LL(k) parsers is studied in Burke and Fisher [1987]. More sophisticated error repair methods are studied in Fischer and LeBlanc [1991] and Lyon [1974].

This chapter did not discuss automatic tools for generating top-down parsers, primarily because the most widely used tool, Yacc, will be discussed in the next chapter. However, good top-down parser generators do exist. One such is called Antlr and is part of the Purdue Compiler Construction Tool Set (PCCTS). See Parr, Dietz, and Cohen [1992] for a description. Antlr generates a recursive-descent parser from an EBNF description. It has a number of useful features, including a built-in mechanism for constructing syntax trees. For an overview of an LL(1) parser generator called LLGen, see Fischer and LeBlanc [1991].